Python Notes: User Input in Python ⌨

Based on the video: User input in Python is easy + exercises ⌨

1. What Is User Input?

MOST IMPORTANT

User input is information provided by a user while a program is running. Python allows us to accept input using the built-in `input()` function.

For example, a program may ask the user to enter their name, age, favourite food, or the price of an item.

The `input()` function

Syntax:

```
input("Prompt message")
```

- `input()` displays an optional prompt.
- The program pauses and waits for the user to enter something.
- The user presses Enter to submit the input.
- The function returns the entered value as a string (`str`).



Example:

```
input("Enter your name: ")
```

Output in the console:

```
Enter your name: Ashwani
```

The program accepts the input, but it is not stored anywhere yet. To use it later, we should assign it to a variable.

Storing user input in a variable

```
name = input("Enter your name: ")
print(name)
```

Output:

```
Enter your name: Ashwani
Ashwani
```

Here:

- `input()` accepts the user's name.
- `name` stores the returned string.
- `print(name)` displays the stored value.

MOST IMPORTANT

Key concept: Use a variable when you need to reuse the user's input later in your program.

2. Displaying User Input Using F-Strings

An f-string allows us to insert variables and expressions directly into a string.

Syntax:

```
f"Text {variable}"
```

Example:

```
name = input("Enter your name: ")
print(f"Hello, {name}!")
```

Output:

```
Enter your name: Ashwani
Hello, Ashwani!
```

You can accept multiple inputs and display them together.

```
name = input("Enter your name: ")
age = input("Enter your age: ")
print(f"Hello, {name}! You are {age} years old.")
```

Example output:

```
Enter your name: Ashwani
Enter your age: 21
Hello, Ashwani! You are 21 years old.
```

Notice that this program displays the age correctly even though the age is still stored as a string.

SHORTCUT

Why? Because an f-string can display strings, integers, floats, and other values without requiring you to convert each value to a string manually.

3. Important Rule: `input()` Always Returns a String

MOST IMPORTANT FREQUENTLY ASKED

This is one of the most important concepts in Python user input.

Even if the user enters a number, `input()` returns a string.

```
age = input("Enter your age: ")
print(age)
print(type(age))
```

If the user enters `21`, the output is:

```
21
<class 'str'>
```

The value is `"21"` as a string, not `21` as an integer.

What happens when we perform calculations?

Consider this code:

```
age = input("Enter your age: ")
age = age + 1
print(age)
```

This produces a `TypeError` because Python cannot concatenate a string and an integer using `+`.

The problem is that:

- `age` is a string, such as `"21"`.
- `1` is an integer.
- Python does not automatically convert `"21"` into `21` for this operation.

Without Type Casting (X)
"21" (str) + 1 (int) → TypeError!

With int(input()) (✓)
int("21") → 21 + 1 = 22 (Valid!)

Solution: Use type casting

Convert the input into a suitable numeric data type before performing arithmetic.

Using `int()`:

```
age = int(input("Enter your age: "))
age = age + 1
print(age)
```

Input:

```
Enter your age: 21
```

Output:

```
22
```

Using `float()`:

```
price = float(input("Enter the price: "))
print(price)
```

Input:

```
Enter the price: 49.99
```

Output:

```
49.99
```

Which conversion function should you use?

Function	When to use it	Example
`int()`	Whole numbers	`int(input("Enter age: "))`
`float()`	Numbers that may have decimal places	`float(input("Enter price: "))`
`str()`	Converting a value into text	`str(21)`
`bool()`	Converting a value into `True` or `False`	`bool(1)`



Remember: Choose `int()` for whole-number input and `float()` when decimal values may be needed.

4. Converting Input in One Line

You do not need to write a separate statement to convert user input after receiving it.

Instead of:

```
age = input("Enter your age: ")
age = int(age)
print(age + 1)
```

You can write:

```
age = int(input("Enter your age: "))
print(age + 1)
```

Both approaches work.

How does the one-line version work?

Python evaluates the nested functions from the inside out:

1. `input()` accepts the user's entry as a string.
2. `int()` converts that string into an integer.
3. The converted value is assigned to `age`.

Step 3: age = [Step 2: int([Step 1: input("Enter your age: ") → "21"]) → 21]
Evaluated from the inside out!

For example:

```
age = int(input("Enter your age: "))
```

If the user enters `"21"`, the stored value becomes the integer `21`.

FREQUENTLY ASKED MEMORY TRICK

Exam tip: Be prepared to explain why `int(input())` or `float(input())` is required when numerical input is used for calculations.

5. Exercise 1: Build a Mad Libs Game

What is Mad Libs?

Mad Libs is a word game in which users supply words such as nouns, verbs, and adjectives. Those words are inserted into a story to create a funny or unexpected result.

This exercise helps you practise:

- The `input()` function.
- Variables.
- F-strings.
- Basic English word categories.
- Combining user-provided text into a story.

Step 1: Understand the word categories

Word type	Meaning	Examples
Adjective	Describes a person, place, or thing	`clean`, `expensive`, `funny`
Noun	Names a person, place, thing, or concept	`teacher`, `zoo`, `pizza`
Verb	Describes an action or state	`run`, `jump`, `scream`

Step 2: Write the complete program

```
# Mad Libs Game
adjective1 = input("Enter an adjective: ")
noun = input("Enter a noun: ")
adjective2 = input("Enter another adjective: ")
verb = input("Enter a verb: ")
adjective3 = input("Enter one more adjective: ")
# Create the story using an f-string
print(f"\nToday I went to a {adjective1} zoo.")
print(f"In an exhibit, I saw a {noun}.")
print(f"The {noun} was {adjective2} and was {verb}ing.")
print(f"I was {adjective3}!")
```

Step 3: Sample execution

```
Enter an adjective: strange
Enter a noun: tiger
Enter another adjective: sleepy
Enter a verb: dance
Enter one more adjective: amazed
Today I went to a strange zoo.
In an exhibit, I saw a tiger.
The tiger was sleepy and was danceing.
I was amazed!
```

The final sentence illustrates an important limitation: simply appending `ing` to every verb does not always produce the correct spelling. For example, `dance` becomes `dancing`, not `danceing`.

You could avoid that spelling problem by asking for a verb ending in `-ing` directly:

```
verb_ing = input("Enter a verb ending in -ing: ")
print(f"The animal was {verb_ing}!")
```

Example: `dancing`, `running`, or `screeching`.

Key takeaway

In this exercise, the user supplies words, the program stores those words in variables, and f-strings insert them into a story.

6. Exercise 2: Calculate the Area of a Rectangle

Formula

The area of a rectangle is calculated using:

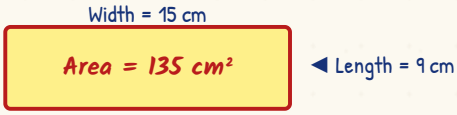
Area = Length × Width

$$\text{Area} = \text{Length} \times \text{Width}$$

For example, if the length is 9 cm and the width is 15 cm:

Area = 9 × 15 = 135 cm²

$$\text{Area} = 9 \times 15 = 135\text{ cm}^2$$



Step 1: Write the program

```
# Rectangle Area Calculator
length = float(input("Enter the length of the
rectangle: "))
width = float(input("Enter the width of the
rectangle: "))
# Calculate the area
area = length * width
# Display the result
print(f"The area is: {area} cm²")
```

Step 2: Sample execution

```
Enter the length of the rectangle: 9
Enter the width of the rectangle: 15
The area is: 135.0 cm²
```

Why do we use `float()`?

The length and width may contain decimal values, such as `9.5` or `15.25`.

Using `float()` permits both whole numbers and decimal numbers.

Without type casting, the input values would be strings, and multiplying two strings is not the correct way to calculate an area.

Key concepts demonstrated

- `float(input(...))` accepts numeric input.
- `*` performs multiplication.
- The `area` variable stores the calculation.
- An f-string displays the result.
- `cm²` represents square centimetres, the unit of area.

MOST IMPORTANT

Important: The unit of area is squared, such as `cm²` or `m²`. It is not simply `cm`.

7. Bonus Exercise: Calculate the Volume of a Cuboid

The video extends the rectangle example by adding a height variable. Strictly speaking, a three-dimensional rectangular solid is called a cuboid.

Formula

Volume = Length × Width × Height

$$\text{Volume} = \text{Length} \times \text{Width} \times \text{Height}$$

Example:

9 × 15 × 7 = 945 cm³

$$9 \times 15 \times 7 = 945 \text{ cm}^3$$

Complete program

```
# Cuboid Volume Calculator
length = float(input("Enter the length: "))
width = float(input("Enter the width: "))
height = float(input("Enter the height: "))
# Calculate volume
volume = length * width * height
print(f"The volume is: {volume} cm³")
```

Sample execution:

```
Enter the length: 9
Enter the width: 15
Enter the height: 7
The volume is: 945.0 cm³
```

Area vs. volume

Feature	Area of rectangle	Volume of cuboid
Formula	Length × Width	Length × Width × Height
Dimensions	2	3
Unit	cm^2	cm^3
Result for 9, 15, 7	135 cm^2 using length and width	945 cm^3

8. Exercise 3: Build a Shopping Cart Program

Objective

Create a program that asks the user for an item, its price, and the quantity they want to purchase. The program then calculates the total cost.

This exercise brings together input, type casting, arithmetic, variables, and formatted output.

Step 1: Identify the variables

Variable	Purpose	Data type
item	Name of the product	str
price	Price of one item	float
quantity	Number of items purchased	int
total	Total cost	float

Step 2: Understand the formula

Total Cost = Price × Quantity

$$\text{Total Cost} = \text{Price} \times \text{Quantity}$$

For example, if one pizza costs ₹4.99 and the customer buys 9 pizzas:

4.99 × 9 = 44.91

$$4.99 \times 9 = 44.91$$

Step 3: Complete program

```
# Shopping Cart Program
item = input("What item would you like to buy? ")
price = float(input("What is the price of one item? ₹"))
quantity = int(input("How many would you like? "))
# Calculate total cost
total = price * quantity
# Display purchase details
print(f"\nYou have bought {quantity} x {item}(s).")
print(f"Your total is: ₹{total:.2f}")
```

Step 4: Sample execution

```
What item would you like to buy? Pizza
What is the price of one item? ₹4.99
How many would you like? 9
You have bought 9 x Pizza(s).
Your total is: ₹44.91
```

Why do we use different data types?

1. Item — `str`

```
item = input("What item would you like to buy? ")
```

The product name is text, so no numerical type conversion is necessary.

2. Price — `float`

```
price = float(input("What is the price? ₹"))
```

Prices may have decimal places, such as `4.99`.

3. Quantity — `int`

```
quantity = int(input("How many would you like? "))
```

The quantity generally represents a whole number, such as `1`, `2`, or `9`.

4. Total — multiplication

```
total = price * quantity
```

Python multiplies the price by the quantity and stores the result in `total`.

9. Rounding and Formatting Decimal Values

In the video, the instructor introduces the built-in `round()` function to limit a result to two decimal places.

Method 1: Using `round()`

Syntax:

```
round(number, ndigits)
```

- `number`: the numeric value to round.
- `ndigits`: the number of decimal places to retain.

Example:

```
total = 44.911
rounded_total = round(total, 2)
print(rounded_total)
```

Output:

```
44.91
```

Another example:

```
print(round(12.3456, 2))  # 12.35
print(round(12.3, 2))     # 12.3
```

Method 2: Using an f-string with `.2f`

For displaying prices, this is especially useful:

```
total = 44.9
print(f"Your total is: ₹{total:.2f}")
```

Output:

```
Your total is: ₹44.90
```

Here, `.2f` formats the number to two decimal places, including trailing zeros.

Code	Output
`round(44.9, 2)`	`44.9`
`f"{44.9:.2f}"`	`"44.90"`
`f"{44.91:.2f}"`	`"44.91"`
`f"{44.916:.2f}"`	`"44.92"`

MOST IMPORTANT

Important distinction: `round()` rounds to the specified number of decimal places. It does not simply truncate all digits beyond that position.

Best practice for this exercise: Use `float()` to accept the price and `f"{total:.2f}"` to display the final amount consistently.

For real financial applications, monetary calculations often use Python's `Decimal` type to handle decimal arithmetic more reliably than binary floating-point numbers.

10. Common Mistakes to Avoid

Mistake 1: Forgetting type casting

```
age = input("Enter age: ")
print(age + 1)
```

Problem: `age` is a string.

Fix: `age = int(input("Enter age: "))`

Mistake 2: Using the wrong numeric type

```
price = int(input("Enter price: "))
```

Problem: A price such as `4.99` cannot be converted using `int()`.

Fix: Use `float()` for prices that may contain decimals.

Mistake 3: Forgetting to store input

```
input("Enter your name: ")
```

Problem: The returned value is not stored for later use.

Fix: `name = input("Enter your name: ")`

Mistake 4: Assuming every entry is valid

```
age = int(input("Enter age: "))
```

Problem: Entering `twenty-one` raises `ValueError`.

Improvement: Validate user input before converting it in programs intended for real users.

11. Quick Revision Sheet

Must-remember concepts

1. `input()` accepts input from the user and returns a string.
2. Variables store the values returned by `input()`.
3. `int(input())` is suitable for whole-number input.
4. `float(input())` is suitable for decimal or whole-number input when a float is needed.
5. F-strings insert variables and expressions into text: `f"Hello, {name}"`.
6. `round(number, 2)` rounds a number to two decimal places.
7. `{total:.2f}` displays a number with exactly two decimal places.

12. Practice Questions

Try solving these yourself before checking the answers.

Test Your Understanding

0/8

1. What does input() return in Python?

An integer by default | **A string** | A float by default | A Boolean

2. Which statement is suitable for accepting an age as a whole number?

`age = input("Age: ")` | `age = float("Age: ")` | **`age = int(input("Age: "))`** | `age = str(input("Age: "))`

3. What is the result of int("25")?

25 as an integer | 25.0 | "25" | ValueError

4. Which data type should be used for a price such as 19.99 in this beginner example?

str | bool | int | **float**

5. What is the formula for a rectangle's area?

Length + Width | **Length × Width** | Length × Width × Height | Length ÷ Width

6. A product costs ₹5 and the customer buys 4. What is the total?

₹9 | ₹1.25 | **₹20** | ₹54

7. What does round(4.567, 2) return?

4.56 | **4.57** | 4.6 | 5

8. What does f"{4.5:.2f}" display?

4.5 | **4.50** | 4.55 | 4

CHECK ANSWERS

13. Mini Project Challenge

After completing the three exercises, try building a Student Information and Marks Calculator.

Your program should:

- Ask for the student's name using `input()`.
- Ask for marks in three subjects using `float(input())`.
- Calculate the total marks and average.
- Display the student's name, total, and average using f-strings.
- Display the average rounded to two decimal places.

Hint:

```
name = input("Enter student name: ")
maths = float(input("Enter Maths marks: "))
python = float(input("Enter Python marks: "))
english = float(input("Enter English marks: "))
total = maths + python + english
average = total / 3
print(f"Student: {name}")
print(f"Total marks: {total}")
print(f"Average: {average:.2f}")
```

MOST IMPORTANT

This mini project combines the most important ideas from the lesson: user input, variables, type casting, arithmetic operators, and formatted output.